



RAJALAKSHMI NAGAR, THANDALAM – 602 105

CP23231 – APPLIED MACHINE LEARNING

LABORATORY

**Laboratory Record Note
Book**

Name	Nithya Shree G V
Branch	ME-CSE
Register No	250711010
Semester	2
Academic Year	2025 -2027



RAJALAKSHMI NAGAR, THANDALAM – 602 105

BONAFIDE CERTIFICATE

Name: Nithya shree G V

Academic Year : 2025 -2027 ODD Semester : 2 Branch: ME-CSE

Register No. 250711010

**Certified that this is the bonafide record of work done by the
above student in the Laboratory during the year**

Signature of Faculty in-charge

Submitted for the Practical Examination held on . . .06.06.2026 . .

Internal Examiner

External Examiner

APPLIED MACHINE LEARNING

LABORATORY MANUAL

Dataset Used:

Manufacturing defects dataset

List of Experiments	
1	Demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples.
2	Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.
3	Write a program to implement k-Nearest Neighbour algorithm to classify the Mushroom data set. Print both correct and wrong predictions.
4	Apply k-Means algorithm to cluster a set of data stored in a CSV file and comment on the quality of clustering.
5	Interpret the results of PCA analysis.
6	Implement the Naïve Bayesian classifier for a sample training data set stored as a CSV file. Compute the accuracy of the classifier, considering few test data sets.

EXPERIMENT 1

Candidate-Elimination Algorithm on Manufacturing Defects

AIM

- To demonstrate the Candidate-Elimination algorithm on the Manufacturing Defects dataset.
- To compute and display the set of all hypotheses consistent with training examples.
- To maintain and converge Specific (S) and General (G) boundary sets throughout training.

ALGORITHM

- Initialize S (most specific hypothesis) and G (most general hypothesis) boundaries.
- For each training example (x, c): if c = positive, generalize S to cover x and remove inconsistent G members.
- If c = negative: specialize G to exclude x, remove any S member inconsistent with x.
- Remove from S or G any hypothesis less general than another in the same set.
- Repeat until all examples are processed.
- Output the final version space defined by S and G boundaries.

SOURCE CODE

Step 1: Load Libraries & Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub, os

path = kagglehub.dataset_download('fahmidachowdhury/manufacturing-defects')
print('Path to dataset files:', path)

csv_file = [f for f in os.listdir(path) if f.endswith('.csv')][0]
df = pd.read_csv(os.path.join(path, csv_file))
print(df.shape)
```

Step 2: Data Exploration

```
df.shape
df.head()
df.tail()
df.dtypes
df.info()
df.describe(include='all').T
print(df.columns.tolist())
```

Step 3: Pre-Processing

3.1 Rename & Clean Columns

```
df.columns = df.columns.str.strip().str.replace(' ', '_')
```

```
df.columns = df.columns.str.lower()
print('Columns:', df.columns.tolist())
```

3.2 Remove Duplicates

```
df.duplicated().sum()
df = df.drop_duplicates()
df.duplicated().sum()
```

3.3 Handle Null Values

```
null = df.isnull().sum()
null[null > 0].sort_values(ascending=False)

def fill_null_values(df):
    df = df.copy()
    for col in df.select_dtypes(include='object').columns:
        if not df[col].mode().empty:
            df[col] = df[col].fillna(df[col].mode()[0])
    for col in df.select_dtypes(include=['int64', 'float64']).columns:
        df[col] = df[col].fillna(df[col].mean())
    return df

df = fill_null_values(df)
print('NULL VALUES ARE FILLED')
```

Step 4: Encoding

4.1 Label Encoding (Target Column)

```
from sklearn.preprocessing import LabelEncoder

label = LabelEncoder()
df['defectstatus'] = label.fit_transform(df['defectstatus'])
print(df['defectstatus'].unique())
```

4.2 One-Hot Encoding (Categorical Features)

```
from sklearn.preprocessing import OneHotEncoder

cat_cols = df.select_dtypes(include='object').columns.tolist()
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
for col in cat_cols:
    encoded = encoder.fit_transform(df[[col]])
    enc_df = pd.DataFrame(encoded,
                          columns=encoder.get_feature_names_out([col]),
                          index=df.index)
    df = pd.concat([df.drop(col, axis=1), enc_df], axis=1)
```

Step 5: EDA — Exploratory Data Analysis

```
df['defectstatus'].value_counts().plot(kind='bar', color=['steelblue', 'tomato'])
plt.title('Defect Status Distribution')
```

```
plt.xlabel('Status'); plt.ylabel('Count')
plt.tight_layout(); plt.show()

plt.figure(figsize=(14, 8))
sns.heatmap(df.corr(numeric_only=True), cmap='coolwarm', annot=False)
plt.title('Feature Correlation Heatmap')
plt.tight_layout(); plt.show()
```

Step 6: Train-Test Split & Scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

x = df.drop('defectstatus', axis=1, errors='ignore')
y = df['defectstatus']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=42, stratify=y)

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

Candidate-Elimination Implementation

```
from sklearn.preprocessing import OrdinalEncoder

ce_df = df.copy()
oe = OrdinalEncoder()
ce_df_enc = pd.DataFrame(oe.fit_transform(ce_df), columns=ce_df.columns).astype(int)

concepts = ce_df_enc.drop('defectstatus', axis=1).values
target = ce_df_enc['defectstatus'].values

def g_0(n): return [['?' for _ in range(n)]]
def s_0(n): return [None]

def is_consistent_with(h, x):
    return all(hi == '?' or hi == xi for hi, xi in zip(h, x))

def candidate_elimination(X, y):
    n = X.shape[1]
    S = [list(X[0]) if y[0]==1 else [None]*n]
    G = g_0(n)
    for xi, yi in zip(X, y):
        if yi == 1:
            G = [g for g in G if is_consistent_with(g, xi)]
            S = [[(si if si == xi[j] else '?') if si is not None else xi[j]]
                  for j, si in enumerate(s)] for s in S]
```

```

else:
    S = [s for s in S if not is_consistent_with(s, xi)]
    new_G = []
    for g in G:
        for j in range(n):
            if g[j] == '?':
                new_g = g[:]
                new_g[j] = xi[j]
                if (any(is_consistent_with(s, new_g) for s in S)
                    and not is_consistent_with(new_g, xi)):
                    new_G.append(new_g)
    G = new_G if new_G else G
return S, G

sample = ce_df_enc.sample(200, random_state=42)
X_ce = sample.drop('defectstatus', axis=1).values
y_ce = sample['defectstatus'].values

S_final, G_final = candidate_elimination(X_ce, y_ce)

print('Specific Boundary (S):')
for s in S_final: print(' ', s)
print()
print('General Boundary (G):')
for g in G_final: print(' ', g)

```

RESULT

The Candidate-Elimination algorithm was successfully implemented on the Manufacturing Defects dataset. The algorithm maintained and converged the Specific (S) and General (G) boundary sets over the training examples. The final version space represents all hypotheses consistent with the given training data, correctly distinguishing defective from non-defective manufacturing conditions.

EXPERIMENT 2

Artificial Neural Network using Backpropagation

AIM

- To build an Artificial Neural Network using the Backpropagation algorithm on the Manufacturing Defects dataset.
- To train the network to detect whether a manufacturing record will result in a defect.
- To evaluate network performance using accuracy and loss metrics.

ALGORITHM

- Initialize network weights randomly; define hidden layers with ReLU activation and output with Sigmoid.
- Forward pass: propagate inputs through layers to compute prediction.
- Compute binary cross-entropy loss between predicted and actual labels.
- Backward pass: compute gradients via chain rule and update weights using Adam optimizer.
- Repeat for a defined number of epochs.
- Evaluate model accuracy on the test set.

SOURCE CODE

Step 1: Load Libraries & Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub, os

path = kagglehub.dataset_download('fahmidachowdhury/manufacturing-defects')
print('Path to dataset files:', path)

csv_file = [f for f in os.listdir(path) if f.endswith('.csv')][0]
df = pd.read_csv(os.path.join(path, csv_file))
print(df.shape)
```

Step 2: Data Exploration

```
df.shape
df.head()
df.tail()
df.dtypes
df.info()
df.describe(include='all').T
print(df.columns.tolist())
```

Step 3: Pre-Processing

3.1 Rename & Clean Columns

```
df.columns = df.columns.str.strip().str.replace(' ', '_')
df.columns = df.columns.str.lower()
print('Columns:', df.columns.tolist())
```

3.2 Remove Duplicates

```
df.duplicated().sum()
df = df.drop_duplicates()
df.duplicated().sum()
```

3.3 Handle Null Values

```
null = df.isnull().sum()
null[null > 0].sort_values(ascending=False)
```

```
def fill_null_values(df):
    df = df.copy()
    for col in df.select_dtypes(include='object').columns:
        if not df[col].mode().empty:
            df[col] = df[col].fillna(df[col].mode()[0])
    for col in df.select_dtypes(include=['int64', 'float64']).columns:
        df[col] = df[col].fillna(df[col].mean())
    return df
```

```
df = fill_null_values(df)
print('NULL VALUES ARE FILLED')
```

Step 4: Encoding

4.1 Label Encoding (Target Column)

```
from sklearn.preprocessing import LabelEncoder

label = LabelEncoder()
df['defectstatus'] = label.fit_transform(df['defectstatus'])
print(df['defectstatus'].unique())
```

4.2 One-Hot Encoding (Categorical Features)

```
from sklearn.preprocessing import OneHotEncoder

cat_cols = df.select_dtypes(include='object').columns.tolist()
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
for col in cat_cols:
    encoded = encoder.fit_transform(df[[col]])
    enc_df = pd.DataFrame(encoded,
                          columns=encoder.get_feature_names_out([col]),
                          index=df.index)
    df = pd.concat([df.drop(col, axis=1), enc_df], axis=1)
```

Step 5: EDA — Exploratory Data Analysis

```
df['defectstatus'].value_counts().plot(kind='bar', color=['steelblue', 'tomato'])
```

```

plt.title('Defect Status Distribution')
plt.xlabel('Status'); plt.ylabel('Count')
plt.tight_layout(); plt.show()

plt.figure(figsize=(14, 8))
sns.heatmap(df.corr(numeric_only=True), cmap='coolwarm', annot=False)
plt.title('Feature Correlation Heatmap')
plt.tight_layout(); plt.show()

```

Step 6: Train-Test Split & Scaling

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

x = df.drop('defectstatus', axis=1, errors='ignore')
y = df['defectstatus']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=42, stratify=y)

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

```

Build & Train ANN

```

from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix

ann = MLPClassifier(
    hidden_layer_sizes=(128, 64),
    activation='relu',
    solver='adam',
    max_iter=300,
    random_state=42,
    learning_rate_init=0.001
)

ann.fit(x_train, y_train)

y_pred_train = ann.predict(x_train)
y_pred_test = ann.predict(x_test)

print('Train Accuracy:', accuracy_score(y_train, y_pred_train))
print('Test Accuracy:', accuracy_score(y_test, y_pred_test))

```

Loss Curve Visualization

```
plt.figure(figsize=(9, 5))
plt.plot(ann.loss_curve_, color='steelblue')
plt.title('Training Loss Curve — ANN (Backpropagation)')
plt.xlabel('Iterations'); plt.ylabel('Loss')
plt.grid(True); plt.tight_layout(); plt.show()
```

Classification Report & Confusion Matrix

```
print('Classification Report:')
print(classification_report(y_test, y_pred_test))

cm = confusion_matrix(y_test, y_pred_test)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Non-Defect', 'Defect'],
            yticklabels=['Non-Defect', 'Defect'])
plt.title('ANN — Confusion Matrix')
plt.ylabel('Actual'); plt.xlabel('Predicted')
plt.tight_layout(); plt.show()
```

RESULT

The Artificial Neural Network with Backpropagation was successfully implemented on the Manufacturing Defects dataset. The network with two hidden layers (128, 64 neurons) converged within the training epochs. The training and test accuracy metrics confirmed the model's capability to detect defective manufacturing conditions. The loss curve showed a steady decrease, demonstrating effective gradient-based optimization.

EXPERIMENT 3

k-Nearest Neighbour (k-NN) Classifier

AIM

- To implement the k-Nearest Neighbour algorithm to classify manufacturing records as defective or non-defective.
- To print both correct and wrong predictions along with their indices.
- To evaluate model accuracy and plot the optimal k value.

ALGORITHM

- Load and preprocess the Manufacturing Defects dataset (encoding, scaling).
- For each test record, compute the Euclidean distance to all training records.
- Select the k nearest neighbors based on the smallest distances.
- Assign the class label by majority vote among the k neighbors.
- Compare predicted label with actual label; print correct and wrong predictions.
- Evaluate using accuracy score and classification report.

SOURCE CODE

Step 1: Load Libraries & Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub, os

path = kagglehub.dataset_download('fahmidachowdhury/manufacturing-defects')
print('Path to dataset files:', path)

csv_file = [f for f in os.listdir(path) if f.endswith('.csv')][0]
df = pd.read_csv(os.path.join(path, csv_file))
print(df.shape)
```

Step 2: Data Exploration

```
df.shape
df.head()
df.tail()
df.dtypes
df.info()
df.describe(include='all').T
print(df.columns.tolist())
```

Step 3: Pre-Processing

3.1 Rename & Clean Columns

```
df.columns = df.columns.str.strip().str.replace(' ', '_')
df.columns = df.columns.str.lower()
print('Columns:', df.columns.tolist())
```

3.2 Remove Duplicates

```
df.duplicated().sum()
df = df.drop_duplicates()
df.duplicated().sum()
```

3.3 Handle Null Values

```
null = df.isnull().sum()
null[null > 0].sort_values(ascending=False)
```

```
def fill_null_values(df):
    df = df.copy()
    for col in df.select_dtypes(include='object').columns:
        if not df[col].mode().empty:
            df[col] = df[col].fillna(df[col].mode()[0])
    for col in df.select_dtypes(include=['int64', 'float64']).columns:
        df[col] = df[col].fillna(df[col].mean())
    return df
```

```
df = fill_null_values(df)
print('NULL VALUES ARE FILLED')
```

Step 4: Encoding

4.1 Label Encoding (Target Column)

```
from sklearn.preprocessing import LabelEncoder

label = LabelEncoder()
df['defectstatus'] = label.fit_transform(df['defectstatus'])
print(df['defectstatus'].unique())
```

4.2 One-Hot Encoding (Categorical Features)

```
from sklearn.preprocessing import OneHotEncoder

cat_cols = df.select_dtypes(include='object').columns.tolist()
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
for col in cat_cols:
    encoded = encoder.fit_transform(df[[col]])
    enc_df = pd.DataFrame(encoded,
                          columns=encoder.get_feature_names_out([col]),
                          index=df.index)
    df = pd.concat([df.drop(col, axis=1), enc_df], axis=1)
```

Step 5: EDA — Exploratory Data Analysis

```
df['defectstatus'].value_counts().plot(kind='bar', color=['steelblue', 'tomato'])
```

```

plt.title('Defect Status Distribution')
plt.xlabel('Status'); plt.ylabel('Count')
plt.tight_layout(); plt.show()

plt.figure(figsize=(14, 8))
sns.heatmap(df.corr(numeric_only=True), cmap='coolwarm', annot=False)
plt.title('Feature Correlation Heatmap')
plt.tight_layout(); plt.show()

```

Step 6: Train-Test Split & Scaling

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

x = df.drop('defectstatus', axis=1, errors='ignore')
y = df['defectstatus']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=42, stratify=y)

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

```

k-NN Implementation

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn.fit(x_train, y_train)

y_pred = knn.predict(x_test)
print('Accuracy:', accuracy_score(y_test, y_pred))

```

Print Correct and Wrong Predictions

```

correct = []
wrong = []

for i, (actual, predicted) in enumerate(zip(y_test, y_pred)):
    if actual == predicted:
        correct.append((i, actual, predicted))
    else:
        wrong.append((i, actual, predicted))

print(f'Total Correct Predictions : {len(correct)}')
print(f'Total Wrong Predictions : {len(wrong)}')

```

```
print('\nSample Correct Predictions (index, actual, predicted):')
for row in correct[:5]: print(row)

print('\nSample Wrong Predictions (index, actual, predicted):')
for row in wrong[:5]: print(row)
```

Find Optimal k using Error Rate

```
error_rate = []
for k in range(1, 21):
    knn_k = KNeighborsClassifier(n_neighbors=k)
    knn_k.fit(x_train, y_train)
    pred_k = knn_k.predict(x_test)
    error_rate.append(1 - accuracy_score(y_test, pred_k))

plt.figure(figsize=(10, 5))
plt.plot(range(1, 21), error_rate, marker='o', color='red')
plt.title('Error Rate vs. k Value')
plt.xlabel('k'); plt.ylabel('Error Rate')
plt.grid(True); plt.tight_layout(); plt.show()

print('Classification Report:')
print(classification_report(y_test, y_pred))
```

RESULT

The k-Nearest Neighbour algorithm was successfully implemented on the Manufacturing Defects dataset with k=5. The classifier correctly identified defective and non-defective manufacturing records. Both correct and wrong predictions were printed with their corresponding indices. The error rate plot helped determine the optimal k value, and the classification report confirmed strong Precision, Recall, and F1 Score metrics for both classes.

EXPERIMENT 4

k-Means Clustering

AIM

- To apply the k-Means algorithm to cluster manufacturing defect records stored in the CSV file.
- To use the Elbow Method to determine the optimal number of clusters.
- To comment on the quality of clustering using the Silhouette Score.

ALGORITHM

- Load and preprocess the Manufacturing Defects dataset (encoding, scaling).
- Standardize all features using StandardScaler.
- Apply the Elbow Method: compute inertia for $k = 1$ to 10 and plot.
- Select optimal k based on the elbow point.
- Fit k-Means with the optimal k ; assign cluster labels.
- Use PCA (2 components) to visualize clusters in 2D.
- Compute Silhouette Score and cross-tab against actual defect class.

SOURCE CODE

Step 1: Load Libraries & Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub, os

path = kagglehub.dataset_download('fahmidachowdhury/manufacturing-defects')
print('Path to dataset files:', path)

csv_file = [f for f in os.listdir(path) if f.endswith('.csv')][0]
df = pd.read_csv(os.path.join(path, csv_file))
print(df.shape)
```

Step 2: Data Exploration

```
df.shape
df.head()
df.tail()
df.dtypes
df.info()
df.describe(include='all').T
print(df.columns.tolist())
```

Step 3: Pre-Processing

3.1 Rename & Clean Columns

```
df.columns = df.columns.str.strip().str.replace(' ', '_')
```

```
df.columns = df.columns.str.lower()
print('Columns:', df.columns.tolist())
```

3.2 Remove Duplicates

```
df.duplicated().sum()
df = df.drop_duplicates()
df.duplicated().sum()
```

3.3 Handle Null Values

```
null = df.isnull().sum()
null[null > 0].sort_values(ascending=False)

def fill_null_values(df):
    df = df.copy()
    for col in df.select_dtypes(include='object').columns:
        if not df[col].mode().empty:
            df[col] = df[col].fillna(df[col].mode()[0])
    for col in df.select_dtypes(include=['int64', 'float64']).columns:
        df[col] = df[col].fillna(df[col].mean())
    return df

df = fill_null_values(df)
print('NULL VALUES ARE FILLED')
```

Step 4: Encoding

4.1 Label Encoding (Target Column)

```
from sklearn.preprocessing import LabelEncoder

label = LabelEncoder()
df['defectstatus'] = label.fit_transform(df['defectstatus'])
print(df['defectstatus'].unique())
```

4.2 One-Hot Encoding (Categorical Features)

```
from sklearn.preprocessing import OneHotEncoder

cat_cols = df.select_dtypes(include='object').columns.tolist()
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
for col in cat_cols:
    encoded = encoder.fit_transform(df[[col]])
    enc_df = pd.DataFrame(encoded,
                          columns=encoder.get_feature_names_out([col]),
                          index=df.index)
    df = pd.concat([df.drop(col, axis=1), enc_df], axis=1)
```

Step 5: EDA — Exploratory Data Analysis

```
df['defectstatus'].value_counts().plot(kind='bar', color=['steelblue', 'tomato'])
plt.title('Defect Status Distribution')
```

```
plt.xlabel('Status'); plt.ylabel('Count')
plt.tight_layout(); plt.show()

plt.figure(figsize=(14, 8))
sns.heatmap(df.corr(numeric_only=True), cmap='coolwarm', annot=False)
plt.title('Feature Correlation Heatmap')
plt.tight_layout(); plt.show()
```

Step 6: Train-Test Split & Scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

x = df.drop('defectstatus', axis=1, errors='ignore')
y = df['defectstatus']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=42, stratify=y)

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

Standardize Features for Clustering

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X = df.drop('defectstatus', axis=1, errors='ignore')
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Elbow Method

```
from sklearn.cluster import KMeans

inertia = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(9, 5))
plt.plot(range(1, 11), inertia, marker='o', color='steelblue')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.title('Elbow Method — Manufacturing Defects')
plt.grid(True); plt.tight_layout(); plt.show()
```

Fit k-Means (K=2) & Visualize

```
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)
df['Cluster'] = clusters

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(9, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap='viridis', alpha=0.6)
plt.xlabel('PC1'); plt.ylabel('PC2')
plt.title('k-Means Clustering — Manufacturing Defects (PCA View)')
plt.colorbar(label='Cluster')
plt.tight_layout(); plt.show()
```

Quality Assessment — Cluster vs. Actual Class

```
crosstab = pd.crosstab(df['Cluster'], df['defectstatus'])
print('Cluster vs DefectStatus Cross-Tabulation:')
print(crosstab)

from sklearn.metrics import silhouette_score
score = silhouette_score(X_scaled, clusters)
print(f'Silhouette Score: {score:.4f}')
```

RESULT

The k-Means clustering algorithm was successfully applied to the Manufacturing Defects dataset. The Elbow Method indicated K=2 as the optimal number of clusters, aligning with the binary classification nature of the dataset (defective vs. non-defective). PCA scatter plots confirmed well-separated clusters, and the cross-tabulation with actual labels showed strong correspondence between cluster assignments and true defect classes. The Silhouette Score confirmed the quality of clustering.

EXPERIMENT 5

Principal Component Analysis (PCA)

AIM

- To interpret the results of PCA analysis applied to the Manufacturing Defects dataset.
- To reduce the high-dimensional feature space to 2 principal components for visualization.
- To compute and interpret the explained variance ratio and cumulative variance.

ALGORITHM

- Load and preprocess the Manufacturing Defects dataset (encoding, scaling).
- Standardize all features using StandardScaler (zero mean, unit variance).
- Apply PCA with n_components=2 to reduce dimensions.
- Plot the 2D scatter of principal components colored by defect status.
- Print the explained_variance_ratio_ to interpret variance captured.
- Plot cumulative explained variance to determine minimum components for 95% variance.
- Interpret the loading/contribution of original features on each principal component.

SOURCE CODE

Step 1: Load Libraries & Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub, os

path = kagglehub.dataset_download('fahmidachowdhury/manufacturing-defects')
print('Path to dataset files:', path)

csv_file = [f for f in os.listdir(path) if f.endswith('.csv')][0]
df = pd.read_csv(os.path.join(path, csv_file))
print(df.shape)
```

Step 2: Data Exploration

```
df.shape
df.head()
df.tail()
df.dtypes
df.info()
df.describe(include='all').T
print(df.columns.tolist())
```

Step 3: Pre-Processing

3.1 Rename & Clean Columns

```
df.columns = df.columns.str.strip().str.replace(' ', '_')
```

```
df.columns = df.columns.str.lower()
print('Columns:', df.columns.tolist())
```

3.2 Remove Duplicates

```
df.duplicated().sum()
df = df.drop_duplicates()
df.duplicated().sum()
```

3.3 Handle Null Values

```
null = df.isnull().sum()
null[null > 0].sort_values(ascending=False)
```

```
def fill_null_values(df):
    df = df.copy()
    for col in df.select_dtypes(include='object').columns:
        if not df[col].mode().empty:
            df[col] = df[col].fillna(df[col].mode()[0])
    for col in df.select_dtypes(include=['int64', 'float64']).columns:
        df[col] = df[col].fillna(df[col].mean())
    return df
```

```
df = fill_null_values(df)
print('NULL VALUES ARE FILLED')
```

Step 4: Encoding

4.1 Label Encoding (Target Column)

```
from sklearn.preprocessing import LabelEncoder
```

```
label = LabelEncoder()
df['defectstatus'] = label.fit_transform(df['defectstatus'])
print(df['defectstatus'].unique())
```

4.2 One-Hot Encoding (Categorical Features)

```
from sklearn.preprocessing import OneHotEncoder
```

```
cat_cols = df.select_dtypes(include='object').columns.tolist()
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
for col in cat_cols:
    encoded = encoder.fit_transform(df[[col]])
    enc_df = pd.DataFrame(encoded,
                          columns=encoder.get_feature_names_out([col]),
                          index=df.index)
    df = pd.concat([df.drop(col, axis=1), enc_df], axis=1)
```

Step 5: EDA — Exploratory Data Analysis

```
df['defectstatus'].value_counts().plot(kind='bar', color=['steelblue', 'tomato'])
plt.title('Defect Status Distribution')
```

```
plt.xlabel('Status'); plt.ylabel('Count')
plt.tight_layout(); plt.show()

plt.figure(figsize=(14, 8))
sns.heatmap(df.corr(numeric_only=True), cmap='coolwarm', annot=False)
plt.title('Feature Correlation Heatmap')
plt.tight_layout(); plt.show()
```

Step 6: Train-Test Split & Scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

x = df.drop('defectstatus', axis=1, errors='ignore')
y = df['defectstatus']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=42, stratify=y)

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

Standardize & Apply PCA

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X = df.drop('defectstatus', axis=1, errors='ignore')
y = df['defectstatus']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

2D Scatter — Visualize Classes in PCA Space

```
plt.figure(figsize=(9, 6))
for lbl in y.unique():
    plt.scatter(X_pca[y == lbl, 0], X_pca[y == lbl, 1],
                label=f'Class {lbl}', alpha=0.6)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA of Manufacturing Defects Dataset')
plt.legend(); plt.tight_layout(); plt.show()
```

Explained Variance Analysis

```

print('Explained Variance Ratio:', pca.explained_variance_ratio_)
print('Total Variance Captured:', pca.explained_variance_ratio_.sum())

pca_full = PCA().fit(X_scaled)
cumvar = pca_full.explained_variance_ratio_.cumsum()

plt.figure(figsize=(9, 5))
plt.plot(cumvar, marker='o', color='steelblue')
plt.axhline(y=0.95, color='r', linestyle='--', label='95% threshold')
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Cumulative Explained Variance vs. Components')
plt.legend(); plt.grid(True); plt.tight_layout(); plt.show()

n_95 = (cumvar >= 0.95).argmax() + 1
print(f'Components needed for 95% variance: {n_95}')

```

Feature Loadings (Component Contributions)

```

feature_names = df.drop('defectstatus', axis=1, errors='ignore').columns

loadings = pd.DataFrame(
    pca.components_.T,
    index=feature_names,
    columns=['PC1', 'PC2']
)

print('Top 10 Features Contributing to PC1:')
print(loadings['PC1'].abs().sort_values(ascending=False).head(10))

print('\nTop 10 Features Contributing to PC2:')
print(loadings['PC2'].abs().sort_values(ascending=False).head(10))

```

RESULT

PCA was successfully applied to the Manufacturing Defects dataset. The first two principal components captured a significant portion of the total variance. The 2D scatter plot showed class separation between defective and non-defective records in the reduced PCA space. The cumulative variance plot was used to determine the minimum number of components required to retain 95% of the variance. Feature loadings revealed which manufacturing metrics contributed most to each principal component.

EXPERIMENT 6

Naïve Bayesian Classifier

AIM

- To implement the Naïve Bayesian classifier on the Manufacturing Defects dataset.
- To compute the accuracy of the classifier on training and test data.
- To validate the model using 5-fold cross-validation and print a classification report.

ALGORITHM

- Load and preprocess the Manufacturing Defects dataset (encoding, scaling).
- Split data into training (80%) and testing (20%) sets with stratified sampling.
- Standardize features using StandardScaler.
- Instantiate GaussianNB and fit on training data.
- Predict on train and test sets; compute Train Accuracy and Test Accuracy.
- Print full Classification Report (Precision, Recall, F1, Support).
- Perform 5-fold Cross-Validation and report mean accuracy.

SOURCE CODE

Step 1: Load Libraries & Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub, os

path = kagglehub.dataset_download('fahmidachowdhury/manufacturing-defects')
print('Path to dataset files:', path)

csv_file = [f for f in os.listdir(path) if f.endswith('.csv')][0]
df = pd.read_csv(os.path.join(path, csv_file))
print(df.shape)
```

Step 2: Data Exploration

```
df.shape
df.head()
df.tail()
df.dtypes
df.info()
df.describe(include='all').T
print(df.columns.tolist())
```

Step 3: Pre-Processing

3.1 Rename & Clean Columns

```
df.columns = df.columns.str.strip().str.replace(' ', '_')
```

```
df.columns = df.columns.str.lower()
print('Columns:', df.columns.tolist())
```

3.2 Remove Duplicates

```
df.duplicated().sum()
df = df.drop_duplicates()
df.duplicated().sum()
```

3.3 Handle Null Values

```
null = df.isnull().sum()
null[null > 0].sort_values(ascending=False)
```

```
def fill_null_values(df):
    df = df.copy()
    for col in df.select_dtypes(include='object').columns:
        if not df[col].mode().empty:
            df[col] = df[col].fillna(df[col].mode()[0])
    for col in df.select_dtypes(include=['int64', 'float64']).columns:
        df[col] = df[col].fillna(df[col].mean())
    return df
```

```
df = fill_null_values(df)
print('NULL VALUES ARE FILLED')
```

Step 4: Encoding

4.1 Label Encoding (Target Column)

```
from sklearn.preprocessing import LabelEncoder
```

```
label = LabelEncoder()
df['defectstatus'] = label.fit_transform(df['defectstatus'])
print(df['defectstatus'].unique())
```

4.2 One-Hot Encoding (Categorical Features)

```
from sklearn.preprocessing import OneHotEncoder
```

```
cat_cols = df.select_dtypes(include='object').columns.tolist()
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
for col in cat_cols:
    encoded = encoder.fit_transform(df[[col]])
    enc_df = pd.DataFrame(encoded,
                          columns=encoder.get_feature_names_out([col]),
                          index=df.index)
    df = pd.concat([df.drop(col, axis=1), enc_df], axis=1)
```

Step 5: EDA — Exploratory Data Analysis

```
df['defectstatus'].value_counts().plot(kind='bar', color=['steelblue', 'tomato'])
plt.title('Defect Status Distribution')
```

```
plt.xlabel('Status'); plt.ylabel('Count')
plt.tight_layout(); plt.show()
```

```
plt.figure(figsize=(14, 8))
sns.heatmap(df.corr(numeric_only=True), cmap='coolwarm', annot=False)
plt.title('Feature Correlation Heatmap')
plt.tight_layout(); plt.show()
```

Step 6: Train-Test Split & Scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
x = df.drop('defectstatus', axis=1, errors='ignore')
y = df['defectstatus']
```

```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=42, stratify=y)
```

```
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

Stratified Train-Test Split (80/20)

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report
```

```
X = df.drop('defectstatus', axis=1, errors='ignore')
y = df['defectstatus']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Naïve Bayes — Fit & Predict

```
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
```

```
y_train_pred = nb_model.predict(X_train)
y_test_pred = nb_model.predict(X_test)
```

```
print('Train Accuracy:', accuracy_score(y_train, y_train_pred))
print('Test Accuracy:', accuracy_score(y_test, y_test_pred))
```

Classification Report

```
print("\nClassification Report:")
print(classification_report(y_test, y_test_pred))
```

5-Fold Cross-Validation

```
X_raw = df.drop('defectstatus', axis=1, errors='ignore')
y_raw = df['defectstatus']

scores = cross_val_score(GaussianNB(), X_raw, y_raw, cv=5)
print('CV Fold Scores:', scores)
print('Mean CV Accuracy:', scores.mean())
print('Std Dev:', scores.std())

print("\nClass Distribution:")
print(y_raw.value_counts())
```

Confusion Matrix Visualization

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_test_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',
            xticklabels=['Pred Non-Defect', 'Pred Defect'],
            yticklabels=['Actual Non-Defect', 'Actual Defect'])
plt.ylabel('Actual'); plt.xlabel('Predicted')
plt.title('Naïve Bayes — Confusion Matrix')
plt.tight_layout(); plt.show()
```

RESULT

The Naïve Bayesian classifier (GaussianNB) was successfully implemented on the Manufacturing Defects dataset. The classifier achieved high accuracy on both training and test sets. The 5-fold cross-validation confirmed consistent model performance with minimal variance across folds. The classification report showed strong Precision, Recall, and F1 Score for both defective and non-defective classes, validating the effectiveness of the Naïve Bayes model for manufacturing defect detection.